

Antigravity & BioNeMo Guide

Complete Universal Pocket Reference — Models, Commands, Tools & 100% Potential

⚡ Golden Rule of Model Economics:

Route high-level architecture to `Claude Opus 5.5` or `Gemini 3.1 Pro`, daily coding to `Gemini 3.8 Flash`, UI styling to `Claude Sonnet 5.5`, and simple scans to `Flash-Lite`.

1. Which Model When? (The 7 Exact Models)

Gemini 3.8 Flash

⚡ ULTRA FAST

The Daily Workhorse (80% of all tasks). Lowest latency, highest tool-calling throughput, and minimal token burn.

Best For: Writing functions, running pytest, CLI benchmarks, high-volume file edits.

Avoid For: Abstract mathematical architecture with 15+ circular dependencies.

Claude Sonnet 5.5

🎨 GOLD STANDARD UI

Unmatched frontend design & clean typed code. Creates stunning, modern, responsive web apps and pristine Python refactors.

Best For: BioNeMo Cockpit UI, modern CSS/JS interfaces, clean architecture.

Avoid For: Trivial shell loops or repetitive regex transforms.

Claude Opus 5.5

💬 MAXIMUM LOGIC

Supreme cognitive depth (S-Tier). Solves novel algorithmic challenges, high-stakes system redesigns, and difficult bug autopsies.

Best For: Designing core algorithms from scratch, multi-module pipeline graphs.

Avoid For: Fast back-and-forth chat or budget-limited sessions.

Gemini 3.1 Pro

💬 STEADY LOGIC

Deliberate multi-hop reasoning. Traces complex concurrency bugs, race conditions, memory leaks, and interface contracts.

Best For: Diagnosing subtle pipeline crashes, security audits, schema boundaries.

Avoid For: Simple one-line edits or trivial renames.

Gemini 3.7 Flash

⚡ STABLE FAST

Rock-solid reliability. Proven schema following and stable tool calls without experimental drift.

Best For: Structured BioNeMo API payloads, routine benchmark profiling.

Gemini 3.6 Flash

 LEGACY FAST

High-volume batch work. Predictable and fast for bulk automated tasks.

Best For: Large-scale regex transforms, linting runs, batch script processing.

GPT-OSS 120B

 OPEN WEIGHT

Unbiased second opinion. Neutral open-source perspective free from proprietary API quirks.

Best For: Neutral code reviews, cross-model verification, fallback when commercial APIs throttle.

2. Interactive Slash Commands (Chat UI)

`/plan`

BLUEPRINT

Creates an interactive, step-by-step blueprint before touching any code. Avoids circular work.

`/goal`

AUTONOMOUS LOOP

Enables persistent, multi-hour execution loops that do not stop until the objective is 100% complete.

`/grill-me`

INTERVIEW

Agent acts as a Staff Engineer, questioning you on edge cases and constraints before coding starts.

`/learn`

MEMORY

Saves corrected behaviors, user rules, and workflows into persistent memory across all future sessions.

`/schedule`

TIMER / CRON

Schedules one-shot timers or recurring cron triggers for background checks without busy-waiting.

3. Core BioNeMo Tools

`bionemo_query_diffdock`

DOCKING

Performs generative molecular docking between protein targets and small-molecule ligands.

`bionemo_query_esmfold`

3D STRUCTURE

Predicts 3D atomic coordinates (.pdb) with pLDDT confidence scores from raw amino acid sequences.

`bionemo_query_esm2_embedding`

EMBEDDINGS

Extracts high-dimensional protein language representations for downstream ML or variant analysis.

`bionemo_sanitize_smiles`

RDKit

Sanitizes ligand SMILES strings with explicit RDKit valence, charge, and aromaticity verification.

`bionemo_validate_sequence`

FASTA

Validates protein sequences against the 20 canonical IUPAC amino acid residues.

`bionemo_fetch_rcsb_pdb`

PDB

Fetches, cleans, and strips water/heteroatoms from experimental crystal structures (e.g. 6LU7, 1HSG).

`bionemo_run_benchmark`

PROFILING

Profiles NIM inference latency, throughput, speedup multipliers, and generates 300 DPI publication plots.

4. The 4 Mastery Levels: Unlocking 100% Potential

● Level 1: The Rookie (0% – 25%)

Stop copy-pasting code into chat. Reference exact file paths (e.g., `src/resistance_engine.py`). Use `/plan` upfront and `/learn` afterwards.

● Level 2: The Intermediate (25% – 50%)

Closed-loop autonomous healing: instruct the agent to edit files, run `pytest`, inspect tracebacks, and fix errors automatically until all pass.

● Level 3: The Pro (50% – 80%)

Multi-MCP chaining: chain RCSB fetch → RDKit sanitization → BioNeMo DiffDock → GitHub Pull Request in a single command. Use `browser_subagent` for automated visual web testing.

● Level 4: The 100% Master (80% – 100%)

Lead Orchestrator Architecture: keep parent context clean. Match the exact model to the problem. Run batch campaigns with `/goal`. Monitor token consumption with `python scripts/token_tracker.py`.

